Stage 2 User Guide — Hover in Wind (ArduPilot SITL + AirSim + Real Drone)

Oct 18, 2026 · @lukas

1. Big picture

Stage 2 takes the Stage 1 hover network and trains it with PPO in gusty wind, so the drone holds position better when pushed. Result in SITL: **13% lower RMSE than Stage 1 in gusts (9 of 10 paired episodes)**, with a small fixed hover offset (~0.2 m) as the trade-off.

Analogy: Stage 1 taught the drone to stand still (copying the teacher). Stage 2 sends it outside on a windy day and lets it practise, keeping what works.

Step	Script	Where	Time	Output
1. Self-test	`drone_hover_disturbance_env.py`	PC, SITL	2 min	2 episodes printed
2. Wind check	`wind_check.py --wind 20`	PC, SITL	2 min	lean angle
3. Train (warm)	`train_stage2_disturbance.py`	PC, SITL	~8.5 h	`hover_disturbance_policy.zip`
4. Test	`test_stage2_policy.py`	PC, SITL	5 min	`test_stage2_warm_<date>.csv/.json`
5. Statistics	`collect_stage2_statistics.py`	PC, SITL	~45 min	`stage2_statistics_<date>.json/.pkl/_report.txt`
6. Summary	`make_stage2_summary.py`	PC only	seconds	`STAGE2_SUMMARY_run1.md`
7. Export	`export_onnx.py`	PC only	seconds	`runs\run1\onnx\warm.onnx`
8. Real drone	`real_fan_test.py`	Raspberry Pi	per test	`real_<model>_fan-<fan>_<date>.csv/.json`
(later) Cold start	`train_stage2_disturbance.py --init cold`	PC, SITL	~8.5 h	comparison only

Status on 18 Oct 2026: steps 1–6 done (warm). Cold start postponed until the hardware is proven. Steps 7–8 are next.

2. Setup

Stage 2 sits next to Stage 1 and reads the Stage 1 network from it. Folder names start with a capital S.

```
drone_recovery_v8_real_hardware\
├── Stage_1\runs\run1\
|   ├── models\stage1_ppo_init.zip        Stage 1 network (warm start +
baseline)
|   ├── models\obs_norm.npz               input scaling (used by Stage 2 and
the real drone)
|   └── results\statistics_summary_*.json Stage 1 numbers (COMPARISON TO
STAGE 1 reads this)
└── Stage_2\
    ├── ap_link.py  pid_expert.py                  (copies of Stage 1's)
    ├── drone_hover_disturbance_env.py
    ├── train_stage2_disturbance.py
    ├── test_stage2_policy.py
    ├── collect_stage2_statistics.py
    ├── make_stage2_summary.py
    ├── wind_check.py
    ├── export_onnx.py                         (PC: model -> .onnx for
the Pi)
    ├── real_fan_test.py                       (Pi: real drone test)
    └── runs\run1\
        ├── models\warm\hover_disturbance_policy.zip, training_info.json,
checkpoints\
        ├── logs\stage2\warm\episodes.csv, learning_curve.png, tb\
        ├── results\   (tests, statistics, summary, real-drone results)
        └── onnx\      (warm.onnx, stage1.onnx)
```

Do not move anything inside `Stage_1\runs\run1` : Stage 2 finds the network, the scaling and the Stage 1 statistics there by path.

Start of every SITL session (GPS mode, same as the Stage 1 guide):

1. Start AirSim (Blocks).
2. In WSL, start SITL from the GPS folder:

```
cd ~/sitl_gps

~/ardupilot/Tools/autotest/sim_vehicle.py -v ArduCopter -f airsim-copter --
console --map --sim-address=172.28.0.1 --out=127.0.0.1:14550
```

3. In MAVProxy, check GPS mode (must show 3):

```
param show EK3_SRC1_POSXY
```

4. In PowerShell:

```
cd
C:\Users\lukas\Desktop\My_Project\AirSim_Project\reinforcement_learning\drone_recovery_v8_real_hardware\Stage_2
```

Why GPS mode: SITL's optical flow is not realistic (Stage 1 guide, optical-flow finding), so all Stage 2 training and tests use GPS. The real drone uses the MTF-01, which is why the real-drone test (section 8) matters.

3. How to run (SITL)

Run these in `Stage_2`, with AirSim and SITL (GPS mode) running, except where it says PC only.

Step 1. Self-test (2 episodes flown by the Stage 1 network, prints wind and distance every 50 steps):

```
py .\drone_hover_disturbance_env.py
```

Step 2. Wind check (proves AirSim wind pushes the ArduPilot drone; prints the lean angle):

```
py .\wind_check.py --wind 20
```

Expected: about 14° lean at 20, about 0.7° at 5. AirSim wind only adds drag, so the setting is not real-world m/s. Match the real fan by **lean angle**.

Step 3. Train, warm start (~8.5 h, 500,000 steps):

```
py .\train_stage2_disturbance.py
```

Check these lines at the start:

- `max |PPO - Stage 1| action difference: 0.00e+00 -> PASS`: the Stage 1 network was loaded.
- `(critic warm-up: actor frozen until 20,000 steps)` and later `(critic warm-up done at 20,480 steps - actor now learning)`.
- A 📊 EPISODE block every 10 episodes (Avg Return, Avg Length, Success Rate, Avg Wind m/s).

At the end: ✅ `TRAINING COMPLETE!` and `hover_disturbance_policy.zip` in `runs\run1\models\warm`.

Step 4. Test (10 episodes in gusts, prints TEST RESULTS and COMPARISON TO STAGE 1):

```
py .\test_stage2_policy.py
```

Other controllers, same seeds: `--model stage1`, `--model pid`, `--model cold`. No wind (offset check): add `--wind-strength 0 --episodes 3`.

Diagnosis only: `--stochastic` samples actions with the training noise (file name gets `_stochastic`). Thesis numbers always use the default, noise-free test.

Step 5. Statistics (~45 min; PID, Stage 1, warm, and cold if trained, all on identical seeds: 10 gust episodes + 2 episodes each at constant 5, 10, 15, 20 m/s):

```
py .\collect_stage2_statistics.py
```

Step 6. Summary (PC only, seconds; console shows aligned tables, the `.md` file opens in VS Code preview with Ctrl+Shift+V):

```
py .\make_stage2_summary.py
```

Later, cold start (comparison "curriculum helps"; after the hardware works):

```
py .\train_stage2_disturbance.py --init cold
```

Then run step 5 again **with all controllers** (no `--models`), so all four are in one statistics file, then step 6.

4. Stop, resume, and watching training

Stop and resume (nothing is lost):

1. Press **Ctrl+C once**. Wait for `💾 Model saved at interruption point: ...disturbance_policy_XXXXX_steps.zip` and for the drone to land.
2. Leave AirSim and SITL running, then:

```
py .\train_stage2_disturbance.py --resume
```

3. Check `✅ PPO model resumed from ... (XXXXX steps)` shows the same number as step 1.

What happens	Saved where	Lost
Ctrl+C	checkpoint at that exact step	nothing
Crash or disconnect (AirSim, SITL)	checkpoint at that exact step, prints ❌ `Training stopped by an error`	nothing

What happens	Saved where	Lost
PC power cut	last automatic checkpoint (every 25,000 steps)	up to ~25 min

On resume, episodes logged after the checkpoint are removed from `episodes.csv` (they are flown again), so the learning curve never counts steps twice. It prints `(removed N episodes logged after the checkpoint - they are re-flown)`.

Safe habits while training or testing runs:

Safe	Not safe
The 📊 EPISODE blocks in the console	Opening `learning_curve.png` or `episodes.csv` (Windows locks the file)
TensorBoard	Selecting text in the PowerShell window (QuickEdit pauses the script; press Esc to release)
Copying the PNG/CSV to the Desktop and opening the copy	Copying output before the `PS C:\...>` prompt is back

Watching training:

- **Console:** 📊 EPISODE block every 10 counted episodes.
- **Picture:** `runs\run1\logs\stage2\warm\learning_curve.png`, redrawn every 2,048 steps from all episodes (return, success, distance). Copy it before opening.
- **TensorBoard:**

```
tensorboard --logdir runs\run1\logs\stage2
```

Then open http://localhost:6006. Use `stage2/return_last20`, `stage2/distance_last20_m`, `stage2/success_rate_last20_pct`, `train/explained_variance`, `train/std`.

Why TensorBoard shows a lower reward: `rollout/ep_rew_mean` (SB3's Monitor) also counts the episodes cut short by each PPO update (`update_pause`). Those are short, so their return is low and pulls the average down: **337** in TensorBoard vs **416.8** in the printout at the end of training. The printout, the CSV and `stage2/return_last20` count complete episodes only, the paper's definition. Use those.

A resume continues the same TensorBoard line (`tb_1`); it does not start `tb_2`.

5. Wind explained

20 m/s is the **limit** of each gust, not the average; the average is about **7 m/s**. Like a speed limit of 110 km/h with an average trip speed of 45.

Wind measure	Value (3,000 simulated episodes of the gust model)
One gust, average	7.0 m/s
One gust, strongest 10%	above 14.5 m/s (top 20)
One episode, average wind	5.2–8.9 m/s for most episodes
One episode, max wind	about 16 m/s on average
Printed Avg Wind (10 episodes)	6.2–7.8 m/s (95% of the time)

Why the average is about 7, not 10:

1. Each gust picks a strength from 0 to 20 and a random 3D direction; the vertical part is scaled by 0.3, so gusts pointing up or down lose most of their strength.
2. The printed Avg Wind averages about 120 gusts (10 episodes × ~12 gusts), so it always lands near 7, like a die rolled 120 times averages near 3.5.

The peaks show in the test printout: `Wind: 6.8m/s (max: 18.4)` per episode and `Maximum Wind Survived` in TEST RESULTS.

The wind model is unchanged from the original AirSim Stage 2 (same gust code), so it is the same in both pipelines.

6. Results so far (run1, warm start)

Stage 2 improves position holding in gusts and beats its own teacher; in calm or steady air it carries a fixed offset of about 0.2 m that Stage 1 had less of and PID does not have.

6.1 Training

Item	Value
Steps / episodes	501,760 / 979 counted episodes
Wall time	about 8.4 h
Return, complete episodes	about 290 at the start → **416.8** (last 50)

Item	Value
TensorBoard `rollout/ep_rew_mean` at the end	337 (includes cut `update_pause` episodes)
Success during training	100% from the first episode (warm start already survives)
Warm-start check	max action difference 0.0e+00 → PASS
Settings	lr 3e-5, std 0.30 (log_std −1.2), ent_coef 0, n_steps 2048, batch 64, epochs 10, target_kl 0.05, critic warm-up 20,000 steps
Environment	500-step episodes at 20 Hz (25 s), start within ±1 m / ±0.3 m, fail at > 4 m, height < 0.5 or > 5 m, tilt > 60°
Reward	+1 − distance − 0.1·speed − 0.01·|action|², −50 on failure

The return stays flat until about 30k steps (critic warm-up, actor frozen to 20,480), then climbs steadily and is still rising slightly at 500k. Success cannot show learning for the warm model (it starts at 100%); return and distance do.

6.2 Gusts 0–20 m/s, identical seeds (10 episodes)

Controller	Success	RMSE (m)	Dist (m)	Held within 0.5 m	End (m)
PID teacher	100%	0.356	0.250	85%	0.256
Stage 1 (BC)	100%	0.372	0.270	84%	0.267
PPO warm	100%	**0.324**	**0.217**	**94%**	0.277

Warm vs Stage 1: RMSE −12.9%; warm wins **9 of 10** paired episodes (episode 8 lost by 0.007 m). Warm vs PID: −9.0%, also 9 of 10. Mean wind 6.80 m/s for all (same seeds).

Max and Worst are not used to compare controllers: Worst was 1.187 m for all three, because the largest distance is the random start point (up to ±1 m), which the seeds make identical.

6.3 Constant wind (2 episodes per level): a fixed offset

Wind	PID end (m)	Stage 1 end (m)	Warm end (m)	Warm RMSE vs Stage 1 (m)
5 m/s	0.01	0.13–0.15	0.26	0.287 vs 0.217

Wind	PID end (m)	Stage 1 end (m)	Warm end (m)	Warm RMSE vs Stage 1 (m)
10 m/s	0.01	0.15	0.20–0.24	0.297 vs 0.271
15 m/s	0.01	0.06–0.14	0.18–0.19	0.236 vs 0.245
20 m/s	0.00	0.11–0.12	0.22–0.25	0.378 vs 0.366

1. Warm settles 0.2–0.25 m from the target, **the same at 5 m/s as at 20 m/s**, so it is a fixed bias in the network, not weakness against wind.
2. Part of it is height: for warm the 3D distance is clearly larger than the horizontal (e.g. 0.239 vs 0.152 m); for PID they are almost equal.
3. PID has no offset because ArduPilot's velocity loop (with its integrator) removes it. The networks output velocity directly and have no integral term.

6.4 Zero-wind check (confirms 6.3)

Controller	Ep 1	Ep 2	Ep 3	Average Dist
Stage 1	0.17	0.13	0.17	0.16 m
PPO warm	0.21	0.18	0.19	0.19 m

With no wind at all, warm still sits further from the target in all 3 episodes. Sanity check: the Stage 1 network gave 0.18 m through the Stage 2 test vs ~0.17 m in the original Stage 1 statistics, so both pipelines measure the same way.

6.5 What this means

- Main claim holds: Stage 2 improves gust rejection (−13% RMSE, 9/10 paired).
- Trade-off: a ~0.2 m steady offset in calm or steady air. Report it.
- Stage 3 is not blocked: flip recovery is about getting upright; 20 cm of hover offset does not matter there.
- Likely cause (not proven): no integral term, and training wind never stops, so settling exactly at zero was never rewarded. Known fixes for later: an error-integral input, or a short fine-tune with calm episodes.

Files: `runs\run1\results\stage2_statistics_<date>.json/.pkl/_report.txt`, `test_stage2_warm_20261018_084915.csv/.json`, `test_stage2_stage1_20261018_085250.csv/.json`, `STAGE2_SUMMARY_run1.md`.

6.6 Summary script output (18 Oct 2026, 09:09)

Check	Status	Key result
Warm start loads Stage 1	PASS	max action difference 0.0e+00
Training, warm start	PASS	501,760 steps, 979 episodes, last 50 success 100%
Training, cold start	NOT RUN	postponed until the hardware works
Warm survives wind	PASS	success 100%, RMSE 0.324 m
Better than Stage 1 in wind	YES	RMSE 0.324 vs 0.372 m (+12.9%)
Warm learns faster than cold	NOT RUN	needs cold

Two lines of that output are not used: **"Steps until rolling success ≥ 80%: 10,741"** (warm succeeds from episode 1, so this is only where the first 20-episode window ends; meaningful for cold only) and the gust table's **Max (m)** (start point, see 6.2).

6.7 Training numbers vs test numbers: quote the test only

The same network looks about twice as good in the training log as in the test:

Gusts, mean wind ~6.9 m/s	Training log, last 50 episodes	Test (statistics)
Horizontal distance	0.105 m	0.217 m
RMSE	0.211 m	0.324 m
End offset	0.097 m	0.277 m
Held within 0.5 m	96%	94%

The cause is not identified (training adds action noise, std ~0.3 m/s; or test episodes differ from training episodes, since PID in the test, 0.250 m, is also worse than warm in training). It does not affect the comparison: PID, Stage 1 and PPO were all flown by the same test, same seeds, noise-free, which is also how the real drone flies.

Rule: in the thesis, quote only test and statistics numbers. Training curves show *that* it learned (return 290 → 417), not *how well* (do not quote 0.14 m or 0.105 m). Optional check if a reviewer asks: `test_stage2_policy.py` vs `test_stage2_policy.py --stochastic` (section 3, step 4).

7. Diagnostic commands

These only read the newest saved statistics file and print; no AirSim needed, nothing is created. Run them in `Stage_2`.

Check A: constant wind, every episode (horizontal vs 3D distance, end offset). If `hdist` is as small as Stage 1 but `dist3d` is bigger, the problem is height; if `end` is the same at every wind level, it is a fixed offset.

```
py -c "import pickle,glob;
f=sorted(glob.glob('runs/run1/results/stage2_statistics_*.pkl'))[-1];
R=pickle.load(open(f,'rb'))['results']; [print('%-7s %5s m/s | start %.2f |
hdist %.3f | dist3d %.3f | rmse %.3f | end %.3f' %
(c,l,e['start_dist'],e['ep_mean_hdist'],e['ep_mean_dist'],e['ep_rmse'],e['ep_
end_dist'])) for c in R for l in R[c]['levels'] for e in R[c]['levels'][l]]"
```

Check B: gusts, episode by episode (is the improvement in most episodes or a few lucky ones?):

```
py -c "import pickle,glob;
f=sorted(glob.glob('runs/run1/results/stage2_statistics_*.pkl'))[-1];
R=pickle.load(open(f,'rb'))['results']; [print('gust ep %2d | start %.2f |
RMSE pid %.3f | stage1 %.3f | warm %.3f' % (i+1,R['warm']['gust'][i]
['start_dist'],R['pid']['gust'][i]['ep_rmse'],R['stage1']['gust'][i]
['ep_rmse'],R['warm']['gust'][i]['ep_rmse'])) for i in range(len(R['warm']
['gust']))]"
```

Zero-wind offset check (about 3 min with AirSim + SITL):

```
py .\test_stage2_policy.py --wind-strength 0 --episodes 3
```

```
py .\test_stage2_policy.py --model stage1 --wind-strength 0 --episodes 3
```

Only the controller differs (warm vs Stage 1); same starts. Ignore the "EXCELLENT … wind" verdict there: it is the original template's verdict from success % only.

Training log, last 50 complete episodes (same measures as the test; result in 6.7):

```
py -c "import csv; r=[x for x in
csv.DictReader(open('runs/run1/logs/stage2/warm/episodes.csv')) if
x['reason']!='update_pause'][-50:]; f=lambda k: sum(float(x[k]) for x in
r)/len(r); print('last 50 training | dist3d %.3f | hdist %.3f | rmse %.3f |
end %.3f | held %.0f%% | wind %.2f' %
(f('mean_dist_m'),f('mean_hdist_m'),f('rmse_m'),f('end_dist_m'),f('hold_pct')
,f('mean_wind_mps')))"
```

8. Real drone test, step by step (indoor, MTF-01)

The test is **indoor**, with the MTF-01 giving position, the same setup your WEMOS already flies PosHold with. GPS is the outdoor backup (Part H). Do the parts in order, and do not start the next step until the **Check** of the current one passes.

How it works: a Raspberry Pi Zero 2 rides on the drone and runs the trained network. You take off in AltHold, switch to PosHold, then flip to GUIDED: the Pi flies for 25 s, then switches the drone back to PosHold itself. Flip away from GUIDED at any time and you fly again.

GUIDED indoors without GPS: ArduPilot allows GUIDED without GPS once it has a position estimate (MTF-01) **and** an EKF origin (ArduPilot: Non-GPS Navigation). The Pi script sends that origin when it starts, so **always start the script before takeoff**.

Part	What	Where	Steps
A	Firmware: add the Pi's serial port, build, flash	WSL	1–4
B	Parameters and mode switch	Mission Planner	5–6
C	Wiring	bench	7
D	Raspberry Pi setup	Pi (SSH)	8–10
E	Export the model, copy it to the Pi	PC + Pi	11–12
F	Link check, props off	Pi	13
G	Fly the test	Pi + drone	14–18
H	GPS backup (outdoor)	—	—

Part A: Firmware (add the Pi's serial port)

The WEMOS has a third UART that is not used yet. One line in `hwdef.dat` turns it into **SERIAL2** for the Pi. Nothing else in your build changes.

Step 1. Find your WEMOS board folder and check the two pins are free (WSL, Ubuntu):

```
cd ~/ardupilot_esp32_build/ardupilot/libraries/AP_HAL_ESP32/hwdef
```

```
ls | grep lukas
```

Pick the folder you flash on the flying WEMOS (below it is called `<wemos-board>`; replace it with your folder name, e.g. `esp32lukasgy91`). Then list every pin it already uses:

```
grep -n "GPIO_NUM" <wemos-board>/hwdef.dat
```

If the list…	Use for the Pi
has neither `GPIO_NUM_21` nor `GPIO_NUM_19`	RX = GPIO21, TX = GPIO19 (the default in this guide)
has `GPIO_NUM_19` (e.g. GY-91 on SPI)	RX = GPIO21, TX = **GPIO22**
has both 19 and 22	stop and send me the list

Check: you know your board folder name and the two pins.

Step 2. Add one line to `hwdef.dat`

```
nano <wemos-board>/hwdef.dat
```

Find the UART lines (they start with `ESP32_SERIAL`). Add the third line **directly under** the two existing ones. Columns are: port, RX pin, TX pin.

```
# --- UARTs: SERIAL0 = USB, UART1 on IO16 (RX) / IO17 (TX) = MTF-01 (SERIAL3)
ESP32_SERIAL  UART_NUM_0  GPIO_NUM_3    GPIO_NUM_1
ESP32_SERIAL  UART_NUM_1  GPIO_NUM_16   GPIO_NUM_17
ESP32_SERIAL  UART_NUM_2  GPIO_NUM_21   GPIO_NUM_19
```

Only the last line is new (use `GPIO_NUM_22` instead of `GPIO_NUM_19` if Step 1 said so). Save with Ctrl+O, Enter, exit with Ctrl+X.

Why it becomes SERIAL2: in ArduPilot's ESP32 code (`HAL_ESP32_Class.cpp`, your commit 9dd2e0f3), SERIAL2 is the **third** `ESP32_SERIAL` line and SERIAL3 the second (your MTF-01). So the new line must be the **last** one, or the MTF-01 moves.

Check: `grep -n "ESP32_SERIAL" <wemos-board>/hwdef.dat` shows exactly three lines, the new one last.

Step 3. Build

```
cd ~/ardupilot_esp32_build/ardupilot
```

```
get_idf
```

```
./waf configure --board=<wemos-board>
```

```
./waf copter
```

Check: the build ends with `'copter' finished successfully`.

Step 4. Flash (your usual WEMOS way: USB to WSL with usbipd)

1. Plug the WEMOS USB into the PC (props off).
2. In **Windows PowerShell as Administrator**, find the WEMOS (CP2104) bus ID and pass it to WSL:

```
usbipd list
```

```
usbipd bind --busid <busid>
```

```
usbipd attach --wsl --busid <busid>
```

3. In WSL, check the port, then flash. **Do not run `erase_flash`**: without it your parameters and calibrations stay.

```
ls /dev/ttyUSB*
```

```
ESPPORT=/dev/ttyUSB0 ESPBAUD=921600 ./waf copter --upload
```

4. Give the USB back to Windows for Mission Planner (PowerShell as Administrator):

```
usbipd detach --busid <busid>
```

Check: flashing ends with `Hash of data verified.` and `Hard resetting`. Connect Mission Planner: parameters are still yours, and the drone still flies **AltHold and PosHold** with the MTF-01 as before. Do this check flight before Part B. If the parameters were reset, redo your usual calibration first.

Part B: Parameters and mode switch (Mission Planner)

Step 5. Turn on SERIAL2 for the Pi. Config → Full Parameter List, set these, then **Write Params** and reboot the drone:

Parameter	Value	Meaning
`SERIAL2_PROTOCOL`	2	MAVLink2 (talks to the Pi)
`SERIAL2_BAUD`	921	921600 baud, same as the Pi script

Nothing else changes: your MTF-01 (SERIAL3), EKF and flow settings stay as they are.

Check: after the reboot, `SERIAL2_PROTOCOL` still reads 2.

Step 6. Put three modes on your mode switch. You take off in **AltHold** (you cannot take off in PosHold), then hover in **PosHold**, then hand over in **GUIDED**. Set your existing mode switch so its three positions are, for example:

Switch position	Mode	Parameter value
1 (e.g. low)	AltHold	2
2 (e.g. middle)	PosHold	16
3 (e.g. high)	Guided	4

Use Config → Flight Modes (or the `FLTMODE1` – `FLTMODE6` parameters that match your switch positions). Keep your usual disarm/emergency method as it is.

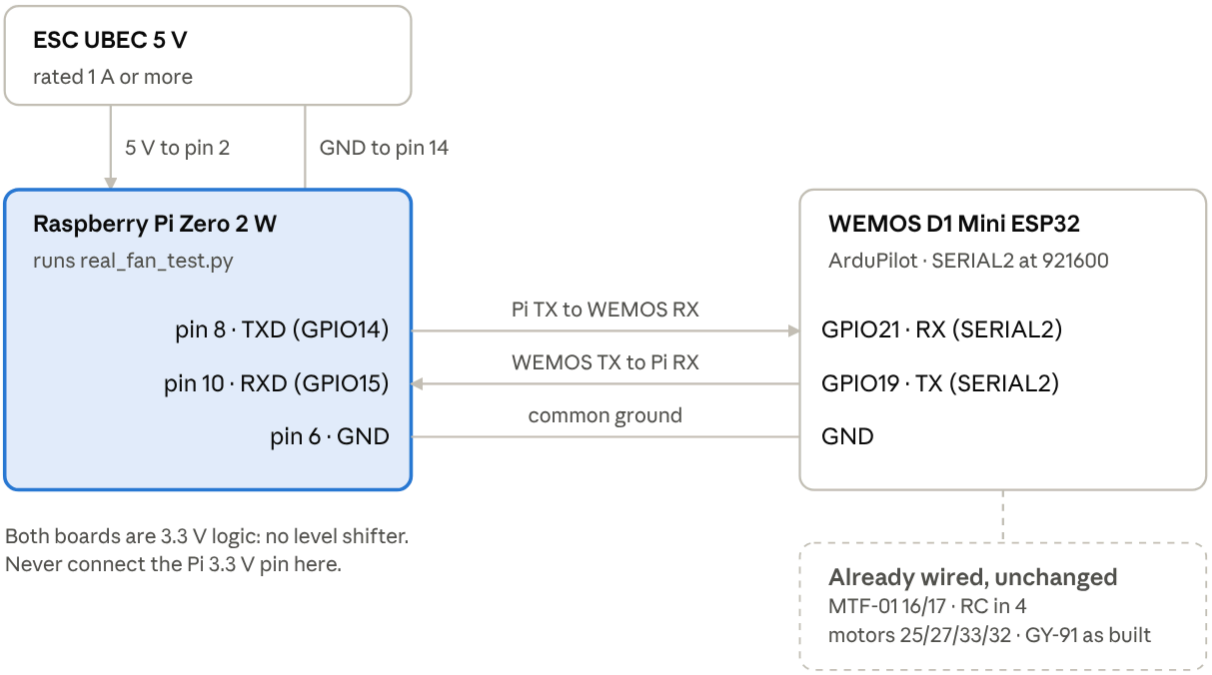
Check: with the radio on, each switch position shows the right mode name on the Mission Planner HUD (props off).

Part C: Wiring

Step 7. Connect the Pi (battery unplugged, props off). Five wires:

Pi pin	Goes to	What
pin 8 (GPIO14, TX)	WEMOS **GPIO21** (RX of SERIAL2)	Pi talks
pin 10 (GPIO15, RX)	WEMOS **GPIO19** (TX of SERIAL2; GPIO22 if Step 1 said so)	Pi listens
pin 6 (GND)	WEMOS GND	common ground
pin 2 (5 V)	UBEC 5 V	Pi power
pin 14 (GND)	UBEC GND	Pi power ground

The Pi talks to ArduPilot over 3 wires on SERIAL2; the UBEC powers it



wiring schematic · Pi Zero 2 to WEMOS SERIAL2, UBEC power

TX always goes to RX. Both boards are 3.3 V logic, so no level shifter. Never connect the Pi's 3.3 V pin. The UBEC must be rated 1 A or more (the Pi takes about 0.3–0.5 A).

Check: before plugging the battery, look again: Pi pin 8 → GPIO21, Pi pin 10 → GPIO19 (or 22), grounds joined.

Part D: Raspberry Pi setup (once)

Step 8. Install the operating system. On the PC, use Raspberry Pi Imager: choose **Raspberry Pi OS Lite (64-bit)**. In its settings (gear icon) set your username (e.g. `lukas`), your Wi-Fi, and turn on SSH. Put the card in the Pi, power it, and connect from PowerShell:

```
ssh lukas@<pi-ip>
```

Check: you get a `lukas@raspberrypi` prompt. (The Pi's IP is in your router's device list.)

Step 9. Turn on the Pi's serial port (on the Pi):

1. Run `sudo raspi-config` → Interface Options → Serial Port → "login shell over serial?" **No** → "serial port hardware enabled?" **Yes** → Finish.
2. Give the fast UART to pins 8/10 (turns Bluetooth off), then reboot:

```
echo "dtoverlay=disable-bt" | sudo tee -a /boot/firmware/config.txt
```

```
sudo systemctl disable hciuart
```

```
sudo reboot
```

Check: after the reboot, `ls -l /dev/serial0` shows `-> ttyAMA0`.

Step 10. Install Python packages (on the Pi, a few minutes):

```
sudo apt install -y python3-venv
```

```
python3 -m venv ~/rlenv
```

```
~/rlenv/bin/pip install numpy onnxruntime pymavlink
```

```
mkdir -p ~/Stage_2/runs/run1
```

Check: `~/rlenv/bin/python -c "import onnxruntime, pymavlink; print('ok')"` prints `ok`.

Part E: Send the model to the Pi

Step 11. Export the networks (PC, PowerShell in `Stage_2`, no AirSim needed):

```
py .\export_onnx.py
```

```
py .\export_onnx.py --model stage1
```

Check: both end with `max |ONNX - PPO| action difference: ... -> PASS` and ✅ `EXPORT COMPLETE`. The files are `runs\run1\onnx\warm.onnx` and `stage1.onnx`.

✏️ **Write down** the line 🧭 `Training heading: __ deg`. In training the drone always faced this way, so on the real drone it must face the same way (±20°). The Pi refuses to fly otherwise.

Step 12. Copy the files to the Pi (PC, PowerShell in `Stage_2`):

```
scp .\ap_link.py .\pid_expert.py .\real_fan_test.py lukas@<pi-ip>:~/Stage_2/
```

```
scp -r .\runs\run1\onnx lukas@<pi-ip>:~/Stage_2/runs/run1/
```

Use the `ap_link.py` from `Stage_2` (it accepts the EKF origin the Pi sends). Your Stage 1 copy is not touched.

Check: on the Pi, `ls ~/Stage_2 ~/Stage_2/runs/run1/onnx` shows the 3 scripts and `warm.onnx`, `stage1.onnx`.



PC to Pi · where every file comes from and where it goes

No new data is needed for the three controllers. Each one comes from a different place:

Controller	Comes from	File needed on the Pi
`pid`	a simple rule inside `pid_expert.py` (fly back toward the target, faster when further away)	none, only `pid_expert.py`
`stage1`	your Stage 1 network, `stage1_ppo_init.zip`	`runs/run1/onnx/stage1.onnx`
`warm`	your Stage 2 network, `hover_disturbance_policy.zip`	`runs/run1/onnx/warm.onnx`

The Pi cannot read the `.zip` models (it has no PyTorch), so `export_onnx.py` turns each one into a small `.onnx` file it can run. On the Pi you then pick the controller with one word: `--model pid`, `--model stage1` or `--model warm`.

Part F: Link check with the props OFF

Step 13. Bench check. Props **off**, battery in, drone on the floor, Pi powered from the UBEC. On the Pi:

```
cd ~/Stage_2
```

```
~/rlenv/bin/python real_fan_test.py --check
```

For 10 s it only **reads** from the drone and sends **nothing** to the motors. It prints once a second: mode, height, velocity, tilt, heading, position age, and what the network **would** send for a pretend target 0.3 m north.

Check (all must pass):

Look at	Pass
last lines	✅ `LINK OK`
`MAVLink packets`	more than 100/s
`pos age`	under about 50 ms
`would send N`	a **positive** number (moves toward the target)
`tilt`	changes when you tilt the drone by hand
`heading`	shows `OK` when you point the drone to the training heading

If you see	Fix
`no heartbeat`	TX/RX swapped (Step 7), `SERIAL2_BAUD` not 921 (Step 5), or Step 9 missed
`no LOCAL_POSITION_NED`	no position yet: the MTF-01 needs to see the floor. Lift the drone ~0.5 m by hand and run again, or check PosHold works in Mission Planner
`TURN THE DRONE`	yaw the drone to the heading from Step 11

Part G: Fly the test (indoor)

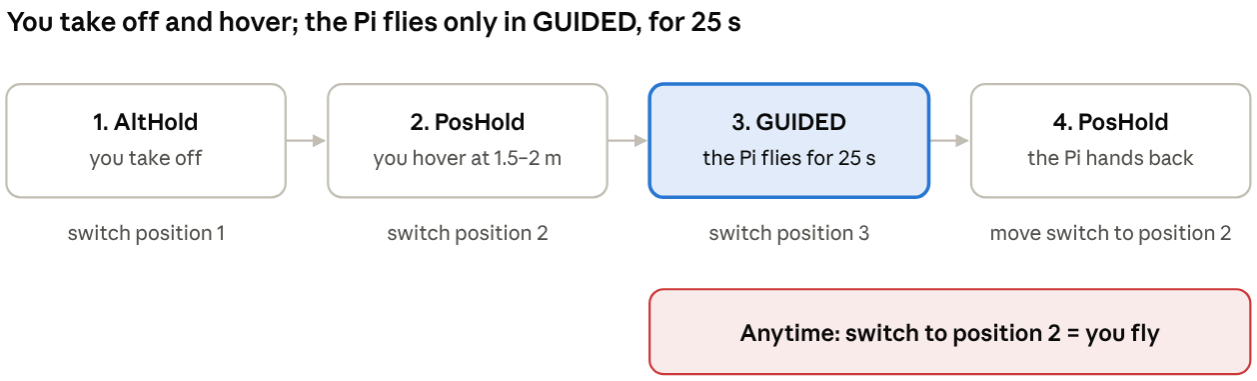
Step 14. Prepare the room. Textured floor for the MTF-01 (a mat or tape if the floor is plain), good light, at least 3 × 3 m clear, nobody under the drone. Mark the hover spot with tape. Fan **off** for now.

Step 15. Start the script BEFORE takeoff (it sends the EKF origin GUIDED needs). Start with `pid`, the simplest controller:

```
~/rlenv/bin/python real_fan_test.py --model pid --fan off --episodes 3
```

Check: it prints `✅ Position and attitude received` and then `⏳ Episode 1: hover in PosHold (sticks centred), then flip the switch to GUIDED...`.

Step 16. Fly one episode



one episode · flight modes and who is flying

How you know the Pi has handed back: each episode always lasts 25 s after `▶️ Pi flying`. Start a stopwatch when you flip to GUIDED and flip to PosHold at about 30 s; or watch the Mission Planner HUD on the laptop (it changes from Guided to PosHold, and says it aloud if speech is on). Flipping to PosHold is always safe: if the Pi already switched, nothing changes; if it is still flying, you take over (that episode is saved as `pilot_takeover`).

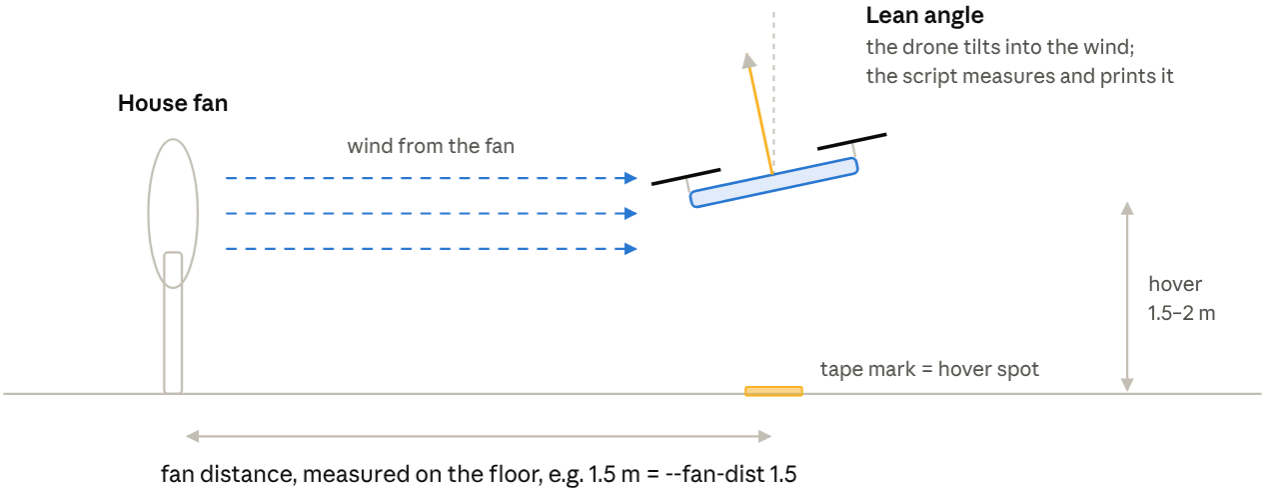
1. Point the drone to the **training heading** (Step 11).
2. Arm and take off in **AltHold**. Climb to about **1.5–2 m** (at least 0.8 m).
3. Switch to **PosHold**. Wait until it holds still, sticks centred.
4. Flip to **GUIDED**. The Pi prints `▶️ Pi flying` and holds the spot where the drone was.
5. Wait **25 s**. The Pi switches the drone to **PosHold** itself and prints the episode line (`Success`, `Dist`, `Tilt`).
6. Move your switch back to **PosHold** (so the switch matches the drone).
7. Next episode: flip to GUIDED again. After the last episode, land in AltHold or PosHold as usual.

🛑 **Take over at any moment:** flip the switch away from GUIDED. The Pi stops at once. The Pi also switches to PosHold by itself at > 1.5 m from the spot, > 0.75 m height error, tilt > 35°, height < 0.4 m, or no position for 0.5 s.

If GUIDED does not engage or the Pi refuses	Fix
Mission Planner says `Need Position Estimate`	the script was not running before takeoff, or the MTF-01 has no position: land, start the script, take off again
Pi prints `heading ... yaw the drone`	yaw to the training heading, flip away and back to GUIDED
Pi prints `height ... climb higher`	climb above 0.8 m, flip away and back

Step 17. The fan test (find the fan settings once, then 4 rounds)

The fan stands a measured distance from the hover mark; the drone leans into its wind



fan test setup · side view

The Pi does not control the fan. You switch it on and set its speed by hand. `--fan` and `--fan-dist` are only **labels** saved in the results, so that later you know which test was which (like writing on a sample bag):

Label	Means	Example
`--fan off`	fan was off	unplugged
`--fan low`	fan on your "low" setting	e.g. speed 1
`--fan high`	fan on your "high" setting	e.g. speed 3
`--fan gusty`	fan on "high" with oscillate on (wind keeps changing)	oscillate button

Label	Means	Example
`--fan-dist 1.5`	fan front stood 1.5 m from the tape mark, measured on the floor	tape measure

Lean angle = how much the drone tilts to fight the wind, like a person leaning into strong wind. The script measures it and prints `Lean (fan strength): mean x°`. We use it instead of the fan's speed number because "speed 3" means something different on every fan and distance, while the lean is the real push the drone feels. It also links to the simulation: AirSim's 20 m/s made the drone lean about 14°.

17a. Find your fan settings (once).

1. Fan on the floor, pointing at the tape mark, **1.5 m** away (front of fan to the mark).
2. Fan on **speed 1**. Fly one episode with PID:

```
~/rlenv/bin/python real_fan_test.py --model pid --fan low --fan-dist 1.5 --episodes 1
```

3. Read the `Lean:` line:

Lean you read	Do this
under 3°	too weak: move the fan closer
3–5°	good **low** setting: write down speed and distance
over 5°	too strong for low: move the fan further away

4. Repeat with **speed 3** and aim for **8–12°**: that is your **high** setting. If the Pi stops the episode (fence), the fan is too strong: move it back.
5. Write both settings down (e.g. low = speed 1 at 1.5 m, high = speed 3 at 1.0 m) and **never change them** afterwards.

17b. The 4 rounds (each command flies 3 episodes; same spot, height and heading every time). Replace `1.5` / `1.0` with your own distances from 17a.

Round 1, fan off:

```
~/rlenv/bin/python real_fan_test.py --model pid --fan off --episodes 3
```

```
~/rlenv/bin/python real_fan_test.py --model stage1 --fan off --episodes 3
```

```
~/rlenv/bin/python real_fan_test.py --model warm --fan off --episodes 3
```

Round 2, fan on your low setting:

```
~/rlenv/bin/python real_fan_test.py --model pid --fan low --fan-dist 1.5 --episodes 3
```

```
~/rlenv/bin/python real_fan_test.py --model stage1 --fan low --fan-dist 1.5 --episodes 3
```

```
~/rlenv/bin/python real_fan_test.py --model warm --fan low --fan-dist 1.5 --episodes 3
```

Round 3, fan on your high setting:

```
~/rlenv/bin/python real_fan_test.py --model pid --fan high --fan-dist 1.0 --episodes 3
```

```
~/rlenv/bin/python real_fan_test.py --model stage1 --fan high --fan-dist 1.0 --episodes 3
```

```
~/rlenv/bin/python real_fan_test.py --model warm --fan high --fan-dist 1.0 --episodes 3
```

Round 4, high setting with oscillate on:

```
~/rlenv/bin/python real_fan_test.py --model pid --fan gusty --fan-dist 1.0 --episodes 3
```

```
~/rlenv/bin/python real_fan_test.py --model stage1 --fan gusty --fan-dist 1.0 --episodes 3
```

```
~/rlenv/bin/python real_fan_test.py --model warm --fan gusty --fan-dist 1.0 --episodes 3
```

Round	Tests	Simulation said
1, off	hover offset with no wind	warm sits ~0.2 m off, PID ~0
2–3, steady fan	steady wind	PID best
4, oscillating	changing wind (gusts)	**warm best: your main claim**

Swap batteries between rounds; add `--notes "battery 2"` if useful.

Step 18. Copy the results to the PC (PowerShell in `Stage_2`):

```
scp lukas@<pi-ip>:~/Stage_2/runs/run1/results/real_* .\runs\run1\results\
```

Per test: `real_<model>_fan-<fan>_<date>.csv` (one line per episode), `.json` (settings + summary), `_steps.csv` (every 20 Hz step: error, velocity, tilt, command, mode). Same measures as SITL: Success = full 25 s without a stop (a takeover counts as a failure), Dist = mean horizontal distance, RMSE, End = last 1 s, Held = time within 0.5 m. Plus real-only: lean angle and Pi loop rate (must be ~20 Hz).

What to expect: the MTF-01 is noisier than SITL's GPS, so distances will be larger than in SITL; compare the controllers **with each other**. The ~0.2 m warm offset (section 6.4) should show with the fan off.

Part H: GPS backup (outdoor)

Use this only if indoor flow does not work out. The scripts do not change.

	Indoor (main)	Outdoor (backup)
Position from	MTF-01	GPS (with your GPS EKF settings)
EKF origin	sent by the Pi script at start	from the GPS fix; the script's origin is ignored
Before GUIDED	script running before takeoff	wait for a GPS 3D fix, script running
Wind	fan only, repeatable	real wind (not repeatable): use `--fan outdoor` and note the weather with `--notes`
Flying	AltHold → PosHold → GUIDED	same

Outdoor numbers are harder to compare between controllers because the wind changes between runs; fly pid, stage1 and warm back to back, and repeat the round.

EKF origin location: indoors the script sends the origin as Kuching (lat 1.5535, lon 110.3593) by default. It only sets where the map shows the drone and the compass declination. Flying elsewhere (e.g. Shah Alam), add `--origin-lat 3.07 --origin-lon 101.50`.

9. Thesis wording

- **Wind:** "Random gusts of 0–20 m/s (AirSim setting), new direction and strength every 1–3 s; mean ≈ 7 m/s, episode peaks up to ≈ 19 m/s. AirSim wind acts as drag only: 20 m/s produces ≈ 14° lean."

- **Main result:** "Stage 2 PPO fine-tuning reduced position RMSE in gusts by 12.9% relative to the Stage 1 network (0.324 vs 0.372 m; better in 9 of 10 paired episodes) and by 9.0% relative to the expert controller, with 100% success."
- **Trade-off:** "In calm or steady wind the fine-tuned policy holds a constant offset of ≈ 0.2 m (Stage 1: ≈ 0.13 m; expert: ≈ 0.01 m), independent of wind strength, consistent with a policy that has no integral action."
- **Reward reporting:** use the complete-episode return (416.8, last 50 episodes), and say that episodes interrupted by policy updates are excluded.
- **Metrics:** Dist/RMSE/Held %; not Max/Worst, which reflect the random start position under identical seeds.
- **Cold start:** not yet run; the "curriculum helps" claim needs it.

10. Troubleshooting

Symptom	Cause	Fix
Training crashed with `OSError: [Errno 22] ... learning_curve.png`	the picture was open in a Windows viewer (file locked)	fixed: training now skips that redraw; still, copy the PNG before opening
"TRAINING COMPLETE" printed after a crash	old bug	fixed: prints ❌ `Training stopped by an error` and saves the model at that step
No reward chart in TensorBoard	old file without the `Monitor` wrapper	fixed; episodes before the fix are in `episodes.csv` and `learning_curve.png`
TensorBoard reward (337) lower than printout (417)	TensorBoard includes `update_pause` episodes	use the printout, CSV or `stage2/return_last20` (section 4)
Script output stops (e.g. after `Observation space: 13`)	QuickEdit: text selected in PowerShell	press Esc; rerun the test if it froze mid-episode
`Stage 1 (no wind): no Stage 1 statistics file found`	`statistics_summary_*.json` moved out of `Stage_1\runs\run1\results`	move it back; do not rerun (keeps the original numbers)

Symptom	Cause	Fix
Summary tables look broken in the console	old version printed raw Markdown	fixed: aligned tables; the `.md` renders in VS Code preview
`'warm' already has results`	protects finished training	`--resume` to continue, `--new-run` for a new experiment
Avg Wind always 6–7 m/s	correct (section 5)	nothing
Pi: `no heartbeat`	wiring (TX/RX swapped), `SERIAL2_BAUD`, or Bluetooth still on the UART	check section 8, steps 5, 7 and 9
Pi: `no LOCAL_POSITION_NED`	EKF has no position	check PosHold works first (flow + rangefinder)
Pi refuses: heading off	drone not facing the training heading	yaw to the heading (section 8, step 11) printed by `export_onnx.py` (±20°)
Pi loop well under 20 Hz	Pi overloaded	close other programs; check `loop_hz` in the results

Training log looks better than the test (0.105 vs 0.217 m): known, see 6.7; quote the test numbers.